

Отчет по лабораторной работе №8

Имитационное моделирование

Mehmet Efe Kantoz

Содержание I

- 1 Цель работы
- 2 Задание
- 3 Теоретическое введение
- 4 Выполнение лабораторной работы
- 5 Выводы

Раздел 1

Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать [`@RUDN_ESystem`] модель.

Раздел 2

Задание

Задание

- Создать рабочий каталог для кода.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.

Раздел 3

Теоретическое введение

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (@process), реализующий его жизненный цикл.

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (@process), реализующий его жизненный цикл.
- Запуск симуляции: ConcurrentSim.run продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый timeout добавляет событие в календарь; когда время

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (@process), реализующий его жизненный цикл.
- Запуск симуляции: ConcurrentSim.run продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый timeout добавляет событие в календарь; когда время
- Сбор статистики: в момент изменения состояния индивида обновляются временные ряды S_a , I_a , R_a , а также запоминается текущее время.

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (@process), реализующий его жизненный цикл.
- Запуск симуляции: ConcurrentSim.run продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый timeout добавляет событие в календарь; когда время
- Сбор статистики: в момент изменения состояния индивида обновляются временные ряды S_a , I_a , R_a , а также запоминается текущее время.
- Завершение: по достижении t_f симуляция останавливается. Все накопленные ряды экспортируются в таблицу.

Общий алгоритм

Программа моделирует стохастическое распространение инфекции в полностью смешивающейся популяции конечного размера. В отличие от детерминированной системы ОДУ, здесь наблюдаются случайные флуктуации, а также возможны ранние вымирания от инфекции при малых начальных I .

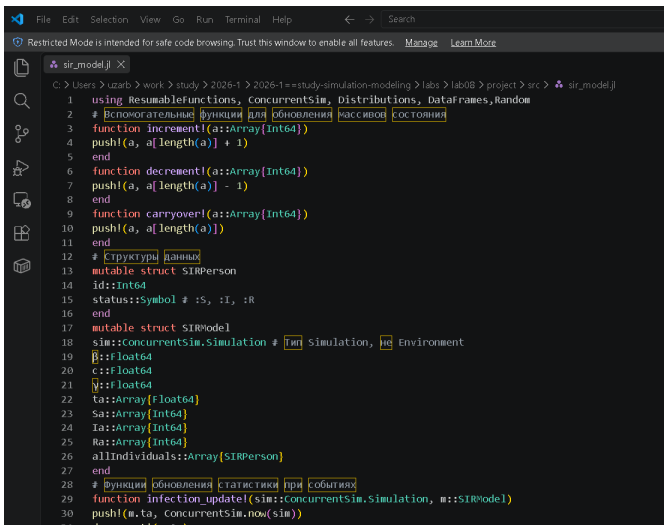
- Инициализация: создание объектов индивидов с начальными статусами, инициализация статистических массивов.
- Планирование процессов: для каждого индивида создаётся параллельный процесс (@process), реализующий его жизненный цикл.
- Запуск симуляции: ConcurrentSim.run продвигает виртуальное время, обрабатывая события в хронологическом порядке. Каждый timeout добавляет событие в календарь; когда время
- Сбор статистики: в момент изменения состояния индивида обновляются временные ряды S_a , I_a , R_a , а также запоминается текущее время.
- Завершение: по достижении t_f симуляция останавливается. Все накопленные ряды экспортируются в таблицу.
- Постобработка: построение графиков, сохранение результатов

Раздел 4

Выполнение лабораторной работы

Создание ядра программы

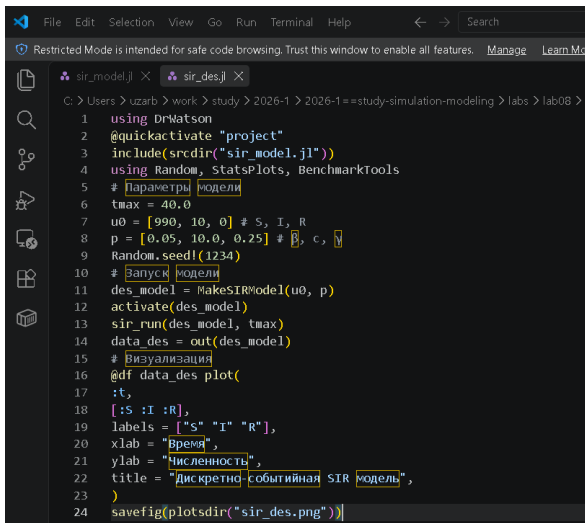
Создаю ядро программы, которое будет использоваться в основном коде (Рисунок 1).



```
1 using ResumableFunctions, ConcurrentSim, Distributions, DataFrames, Random
2 # Вспомогательные функции для обновления массивов состояний
3 function increment!(a::Array{Int64})
4     push!(a, a[length(a)] + 1)
5 end
6 function decrement!(a::Array{Int64})
7     push!(a, a[length(a)] - 1)
8 end
9 function carryover!(a::Array{Int64})
10    push!(a, a[length(a)])
11 end
12 # Структуры данных
13 mutable struct SIRPerson
14     id::Int64
15     status::Symbol # :S, :I, :R
16 end
17 mutable struct SIRModel
18     sim::ConcurrentSim.Simulation # Тип Simulation, не Environment
19     β::Float64
20     c::Float64
21     γ::Float64
22     ta::Array{Float64}
23     Sa::Array{Int64}
24     Ia::Array{Int64}
25     Ra::Array{Int64}
26     allIndividuals::Array{SIRPerson}
27 end
28 # Функции обновления статистики при событиях
29 function infection_update!(sim::ConcurrentSim.Simulation, m::SIRModel)
30     push!(m.ta, ConcurrentSim.now(sim))
31     decrement!(m.Sa)
```

Основной код модели

Создаю основной код с базовым прогоном модели (Рисунок 2).



```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

sir_model.jl x sir_des.jl x

C:\> Users \uzarb > work > study > 2026-1 > 2026-1==study-simulation-modeling > labs > lab08 >

1 using DrWatson
2 @quickactivate "project"
3 include(sourcedir("sir_model.jl"))
4 using Random, StatsPlots, BenchmarkTools
5 # Параметры модели
6 tmax = 40.0
7 u0 = [990, 10, 0] # S, I, R
8 p = [0.05, 10.0, 0.25] #  $\beta$ , c,  $\gamma$ 
9 Random.seed!(1234)
10 # Запуск модели
11 des_model = MakeSIRModel(u0, p)
12 activate(des_model)
13 sir_run(des_model, tmax)
14 data_des = out(des_model)
15 # Визуализация
16 @df data_des plot(
17 :t,
18 [:S :I :R],
19 labels = ["S" "I" "R"],
20 xlab = "Время",
21 ylab = "Численность",
22 title = "Дискретно-событийная SIR модель",
23 )
24 savefig(plotsdir("sir_des.png"))
```


Запуск базового прогона

Запускаю базовый прогон (Рисунок 3).

```
r further precompilation to work with the unexpected versions.  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project> julia scripts/sir_des.jl  
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project'  
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project> |
```

Рисунок 3: Запуск

Генерация форматов (1)

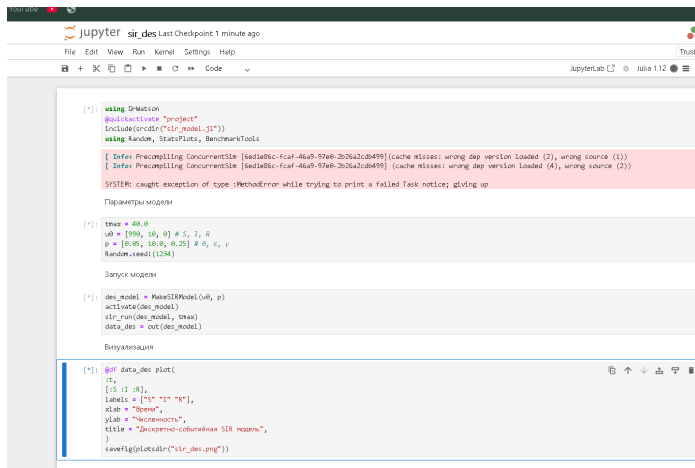
Создаю все производные форматы (Рисунок 4).

```
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project'
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project> julia scripts/tangle.jl script
s/sir_des.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project'
Генерация из: scripts/sir_des.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08
\project\scripts\sir_des.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\scripts
\sir_des\sir_des.jl'
✓ Чистый скрипт: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\scripts\sir_des/
sir_des.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\pro
ject\scripts\sir_des.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\markdow
n\sir_des\sir_des.qmd'
✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\markdown\sir_des/sir_de
s.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\
scripts\sir_des.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\noteboo
ks\sir_des\sir_des.ipynb'
✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\notebooks\sir_des/sir
_des.ipynb
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project> |
```

Рисунок 4: Производные форматы

Запуск Notebook

Запускаю файл notebook (Рисунок 5).



The screenshot shows a Jupyter Notebook window titled "sir_des Last Checkpoint 1 minute ago". The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The notebook content is organized into sections: a code cell for package management and model parameter setup, a text cell for parameter labels, another code cell for model initialization and execution, a text cell for visualization, and a final code cell for plotting and saving the figure.

```
[*]: using DrWatson
@quickactivate "project"
include(sourcedir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools

[ Inference Precompiling ConcurrentSIR [5ed1e86c-fcaf-46a9-97e0-2b26a2c0b499] (cache misses: wrong dep version loaded (2), wrong source (1))
[ Inference Precompiling ConcurrentSIR [5ed1e86c-fcaf-46a9-97e0-2b26a2c0b499] (cache misses: wrong dep version loaded (4), wrong source (2))
SYSTEM: caught exception of type :MethodError while trying to print a failed Task notice; giving up

Параметры модели

[*]: tmax = 40.0
u0 = [999, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] # β, ε, γ
Random.seed!(1234)

Запуск модели

[*]: des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)

Визуализация

[*]: @df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlabel = "Время",
    ylabel = "Численность",
    title = "Дискретно-событийная SIR модель",
)
savefig(plotdir("sir_des.png"))
```

Рисунок 5: notebook

Результат моделирования

В результате получаю следующий график (Рисунок 6).

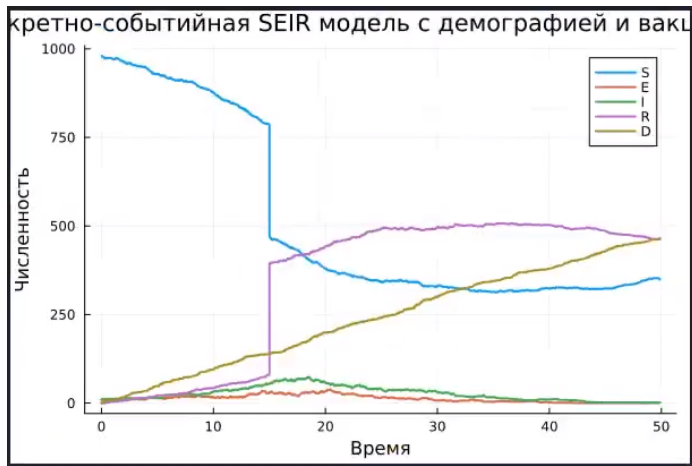


Рисунок 6: График модели

Модификация параметров

Далее модифицирую код, в котором задаю разные значения для модели и запускаю его (Рисунок 7).

```
(\lab08\project"
=== Запуск базового эксперимента SEIR ===
[Событие] Вакцинация проведена в t = 15.0. Вакцинировано: 314

=== Сравнение стохастического и детерминированного выздоровления ===

=== Анализ чувствительности параметров (Тестирование вариаций  $\beta$ ) ===
Параметр  $\beta = 0.03$  | Пик заболеваемости (I): 31 в момент времени t = 20.812687156323367
Параметр  $\beta = 0.05$  | Пик заболеваемости (I): 111 в момент времени t = 31.468917750178672
Параметр  $\beta = 0.08$  | Пик заболеваемости (I): 233 в момент времени t = 19.833061835579347

=== Запуск бенчмарка производительности для крупной популяции ===
BenchmarkTools.Trial: 10000 samples with 7 evaluations per sample.
Range (min - max):  4.014  $\mu$ s - 33.380  $\mu$ s | GC (min - max): 0.00% - 0.00%
Time (median):      11.400  $\mu$ s | GC (median):  0.00%
Time (mean  $\pm$   $\sigma$ ): 12.190  $\mu$ s  $\pm$  5.016  $\mu$ s | GC (mean  $\pm$   $\sigma$ ): 0.00%  $\pm$  0.00%

Histogram: frequency by time
4.01  $\mu$ s 33.3  $\mu$ s
Memory estimate: 480 bytes, allocs estimate: 16.
Тестирование успешно завершено. Все графики сохранены в директории 'plots/'.
```

Рисунок 7: Новый код

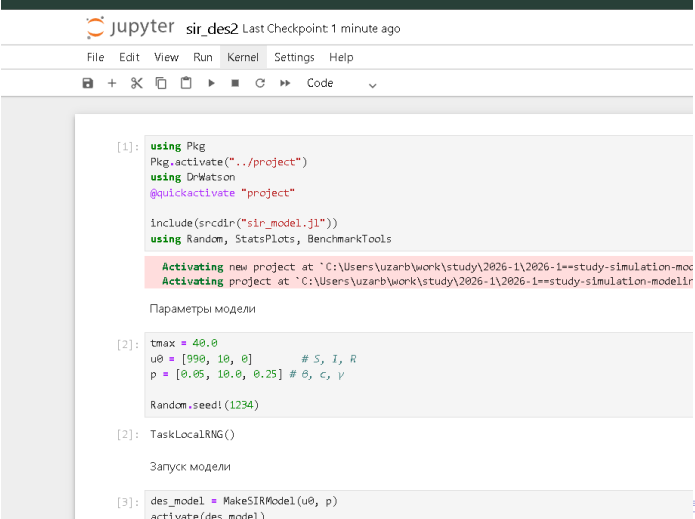
Создаю все производные форматы (Рисунок 8).

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab08\project\scripts> julia scripts/tangle.jl  
l scripts/sir_des2.jl
```

Рисунок 8: Производные форматы

Запуск измененного Notebook

Запускаю файл notebook (Рисунок 9).



The screenshot shows a Jupyter Notebook interface with the title 'sir_des2' and a status 'Last Checkpoint 1 minute ago'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and execution. The notebook content consists of three code cells:

```
[1]: using Pkg
      Pkg.activate("../project")
      using DrWatson
      @quickactivate "project"

      include(sourcedir("sir_model.jl"))
      using Random, StatsPlots, BenchmarkTools

      Activating new project at `C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-moc
      Activating project at `C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modelir

      Параметры модели

[2]: tmax = 40.0
      u0 = [990, 10, 0] # S, I, R
      p = [0.05, 10.0, 0.25] # θ, c, γ

      Random.seed!(1234)

[2]: TaskLocalRNG()

      Запуск модели

[3]: des_model = MakeSIRModel(u0, p)
      activate(des_model)
```

The output of the first cell shows the activation of the project and the inclusion of the model file. The output of the second cell shows the parameters of the model. The output of the third cell shows the creation of the model object.

Итоговые графики

В результате получаю следующие графики (Рисунок 10).

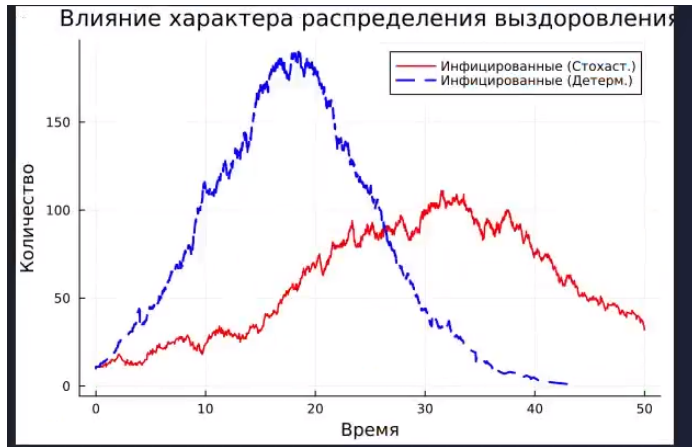


Рисунок 10: График модели

Раздел 5

Выводы

После выполнения данной лабораторной работы мы изучили дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR.

Раздел 6

Список литературы

Список литературы